

Week 3 Lab Report

Course: Introduction to Big Data Analytics

Student: shimwe muhire johnson

This lab worked with a class list of 40 students stored in a CSV file. The goal was to practice three things: making decisions with if/elif/else, repeating work with loops, and packaging logic into reusable functions. The last and biggest part of the lab was building a small pipeline that reads the raw file, grades every student, and copes with the fact that real data is sometimes messy.

Out of the 40 students, 3 had a score that was not a real number — text like "N/A", "absent", or "--" instead of a mark. In this dataset those were students AUCA004, AUCA021, and AUCA038. The other 37 students had a normal numeric score and could be graded as usual.

Instead of assuming every score would convert cleanly to a number, each row was processed inside a try/except block. The code attempted `int(score)` for every student; when that worked, the score was added to the running total and the student was counted as valid. When it failed — because the value was text, not a number — Python raised a `ValueError`, which was caught instead of allowed to crash the program. The failing student's ID was recorded in a "bad_ids" list and a bad-record counter was increased, and the loop simply moved on to the next row. This meant one messy row never stopped the other 39 from being processed, and nothing was silently lost — every skipped student was named and counted, not just dropped without a trace.

Among the 37 valid students, the average score was 65.9, and the class split was 28 passing (score 50 or above) versus 9 failing. The best-performing student scored 96 and earned an A grade. Breaking the results down by district, Muhanga had the highest average score at 71.4, while Nyarugenge had the lowest at 51.2 — a reminder that a class-wide average can hide real differences between smaller groups.

The main lesson was that real datasets are rarely perfectly clean, and code that assumes otherwise is fragile. A try/except block let the pipeline treat bad data as an expected possibility rather than a fatal error, while still keeping a clear record of what was skipped and why. That combination — resilience plus transparency — feels like the right default for any pipeline that processes data it doesn't fully control.